Custom `rm` Utility in x86-64 Linux Assembly

A lightweight, dependency-conscious implementation of the classic UNIX `rm` file/directory removal utility. Written in raw **x86-64 assembly** for Linux, using Netwide Assembler (NASM) syntax and linking with the standard C library for auxiliary string operations.

This utility features:

- Core file deletion (`sys_unlink` syscall).
- Directory removal (`sys_rmdir` syscall).
- Command-line flag parsing (`-v` for verbose output, `-i` for interactive confirmation).
- Explicit metadata examination using `sys_newfstatat` to safely distinguish between regular files and directories.

🏗️ Memory Layout & Program Anatomy

Assembly programs organize memory space using structural **directives**. This codebase is segmented into three distinct functional divisions:

1. `.data` Section (Initialized Data)

This section contains pre-defined constants and hardcoded string patterns. Because these are initialized, they reside permanently inside the compiled binary executable.

```
section .data
    v_flag_str db "-v" , 0
    i_flag_str db "-i" , 0
    usage_msg db "the usage: ./rm <file1> <file2>...", 0
    usage_msg_len equ $ - usage_msg
    deleted_msg db "deleted :", 0
    deleted_msg_len equ $ - deleted_msg
    fstat_fail_msg db "fstat fail", 0
    fstat_fail_len equ $ - fstat_fail_msg
    newline db 10
    question_msg db "do you to delete this :" , 0
    question_msg_len equ $ - question_msg
```

- `db` **(Define Byte)**: Allocates 1 byte of memory per character. Every string is terminated with a `, 0` (null-terminator) to guarantee compatibility with external C library functions like `strcmp`.
- `equ $ - label` : An assembler macro that automatically evaluates the length of a string constant at compile time.
  - The special symbol `$` evaluates to the *current address* of the program counter.
  - Subtracting the starting memory address of the string (`label`) yields its exact length.

2. `.bss` Section (Uninitialized Data)

This block reserves empty memory units inside the system RAM at runtime. It does not bloat the size

of the compiled binary on disk.

```
section .bss
    response resb 2
    v_flag resb 1
    i_flag resb 1
```

- `resb` **(Reserve Byte)**: Allocates the specified number of bytes.
  - `response` is given 2 bytes to safely buffer user feedback (1 byte for `'y'` / `'n'` and 1 byte for the newline key representation).
  - `v_flag` and `i_flag` act as 1-byte booleans (implicitly defaulting to 0).

3. `.text` Section (Executable Logic)

This is where the actual assembly instructions run. By default, it is configured as read-only memory by the operating system kernel to prevent memory safety exploits.

🚀 Step-by-Step Code Walkthrough

1. Bootstrapping and Environment Initialization (`_start`)

When Linux initiates your program, the operating system lays out the program arguments on top of the **Stack** (`rsp` pointer).

[Linux Stack Frame Startup State]

- `[rsp]` contains `argc` (integer count of arguments).
- `[rsp + 8]` contains `argv[0]` (pointer to the program name string).
- `[rsp + 16]` contains `argv[1]` (pointer to the first user-passed argument string).

```
global _start
_start:
    mov rdi , [rsp]      ; 1. Load the argc integer value into RDI
    lea rsi , [rsp + 16] ; 2. Compute the starting address of argv[1] into RSI
    cmp rdi , 1          ; 3. Did the user execute with no files/flags?
    je print_usage       ; 4. If RDI == 1, jump to display help message
    call main            ; 5. Proceed to core program execution block
```

- **Register Setup**: In 64-bit Linux calling conventions, parameters are passed to functions inside dedicated CPU registers: `rdi` (1st argument) and `rsi` (2nd argument).
- `[rsp]` **vs** `lea` : Bracket syntax `[rsp]` fetches the *contents* at the top of the stack (the count). `lea` (Load Effective Address) computes the physical memory address offset without reading the value inside, letting us pass the argument array pointer to `main` .

2. Main Entry Function Frame (`main`)

```
main:
    push rbp
    mov rbp , rsp
    sub rsp  , 208       ; Allocate 208 bytes of scratch stack memory workspace
```

- **Prologue**: Saving the previous base pointer (`push rbp`) and copying the stack pointer (`mov rbp, rsp`) creates a clean stack frame.
- **Stack Allocation**: Subtracting 208 bytes from the stack pointer (`rsp`) reserves a safe scratchpad area. This ensures we have enough space to house the 144-byte Linux kernel `stat` metadata structure.

```
    mov [rbp - 8] , rdi  ; Safely preserve argc on the stack
    mov [rbp - 16] , rsi ; Safely preserve argv on the stack

    mov rdi , [rbp - 8]  ; Reload parameters
    mov rsi , [rbp - 16]
    call check_flags     ; Scan command line variables for active flags
```

- **Preserving Register States**: General-purpose registers are easily modified by external function calls. Saving `rdi` and `rsi` inside stack-allocated offsets `[rbp - 8]` and `[rbp - 16]` protects our command-line configuration from corruption.

3. Pointer Offset Arithmetic

```
    mov r12 , [rbp - 16] ; Load the base argv array pointer into R12
    imul rax , rax , 8   ; Multiply the flag count returned in RAX by 8 bytes
    add r12 , rax        ; Shift R12 forward past those flags
```

- `imul rax, rax, 8` : In a 64-bit environment, memory addresses (like individual string pointers in `argv`) are exactly 8 bytes long.
- **The Math**: `check_flags` returns the total number of flagged options parsed. By multiplying this integer by 8, we obtain the precise byte offset needed to shift our active pointer `r12` directly to the start of the actual file targets.

4. File Information Querying (`delete_loop`)

```
delete_loop:
    mov rax , [r12]      ; Dereference R12 to get the address of the current file
    cmp rax , 0          ; Is it a null pointer?
    je delete_end        ; If yes, terminate execution
```

- Linux terminates the argument vector pointer array (`argv`) with a `0` value. Checking for this null-pointer detects when the program has evaluated all arguments.

```
; Execute fstatat system call
mov rax , 262        ; Syscall 262: sys_newfstatat
mov rdi , -100       ; AT_FDCWD: Target the current working directory
mov rsi , [r12]      ; Target filename string address pointer
lea rdx , [rsp + 48] ; Target our allocated stack buffer space for metadata w
mov r10 , 0x100      ; AT_SYMLINK_NOFOLLOW: Do not resolve symlinks
syscall
```

- `syscall 262` : Talks to the kernel to inspect file descriptors. It populates a `stat` structure within the stack region we offset using `lea rdx, [rsp + 48]` .

5. File System Class Isolation

```
mov eax , dword [rdx + 24] ; Extract file mode bitmask (24 bytes into stat st
and rax , 0xF000           ; Isolate type definition bytes
cmp rax , 0x8000           ; Does it match 0x8000 (S_IFREG)?
je this_is_file
cmp rax , 0x4000           ; Does it match 0x4000 (S_IFDIR)?
je this_is_dir
```

- **Bitmasking (** `and rax, 0xF000` **):** Files contain permissions (like read, write, and execute) encoded inside their mode flags. Masking with `0xF000` strips away permission profiles to isolate only the system identity code:
  - `0x8000` defines standard data **files**.
  - `0x4000` defines structural **directories**.

6. System Destructors (`this_is_file` & `this_is_dir`)

Both sections compare flags and check user input before calling the kernel to delete the target.

```
; File Destruction
delete_file:
    mov rax , 87         ; Syscall 87: sys_unlink
    mov rdi , [r12]      ; Pointer to filename string
    syscall

; Directory Destruction
delete_dir:
    mov rax , 84         ; Syscall 84: sys_rmdir
    mov rdi , [r12]      ; Pointer to directory path string
    syscall
```

7. Core Subroutines

Prompt Engine (`check_answear`)

Handles the interactive flag logic (`-i`).

```
check_answear:
    ; ... (stdout write calls to print the prompt messages)

    ; Read user reply
    mov rax , 0          ; Syscall 0: sys_read
    mov rdi , 0          ; File Descriptor 0 (stdin - keyboard)
    mov rsi , response   ; Buffer location address
    mov rdx , 2          ; Read up to 2 bytes max
    syscall

    cmp byte [response] , 'y' ; Did user press 'y'?
    je set_rax_1
    jmp set_rax_0
```

Flags Parsing Engine (`check_flags`)

Iterates through command-line parameters to configure global execution states.

```
check_loop:
    cmp r14 , r9         ; Compare current loop index (R14) to total argc (R9)
    je check_done

    mov rdi , [r12]      ; Param 1 for strcmp: current argv string
    mov rsi , i_flag_str ; Param 2 for strcmp: "-i"
    call strcmp
    cmp rax, 0           ; Zero means strings match perfectly
    je set_i_flag
```

🛠️ Compilation and Linking Instructions

Because this binary utilizes external functions defined inside the standard C library (`strcmp` and `strlen`), it must be compiled into an ELF64 object block and linked using `gcc` .

1. Assemble with NASM

Converts your assembly source file into an ELF64 machine-code object structure:

```
nasm -f elf64 main.asm -o main.o
```

2. Link with GCC

Links with the shared standard C library (`libc`) and sets up execution mappings:

```
gcc -no-pie main.o -o my_rm
```

(Note: `-no-pie` disables Position Independent Executables, which is required to link classic manual entry points like `_start`).

3. Test Deletion

```
# Create dummy parameters to test safely
touch dummy.txt
mkdir -p dummy_dir

# Delete with interactive confirm (-i) and verbose output (-v)
./my_rm -i -v dummy.txt dummy_dir
```